

머신러닝 기초 보고서, 김도훈, 2026406041

<목차>

1. 머신러닝
 - 1-1 머신러닝의 종류(지도 학습, 비지도 학습, 강화 학습)
2. 퍼셉트론
SUB PyTorch란 무엇인가
3. 활성화 함수
 - 3-1 step 함수 및 PyTorch 사용 예
 - 3-2 sigmoid 함수 및 PyTorch 사용 예
 - 3-3 Tanh 함수 및 PyTorch 사용 예
 - 3-4 ReLU 함수 PyTorch 사용 예
 - 3-5 Leaky ReLU 함수 PyTorch 사용 예
4. 비용함수(Loss Function, **cost function**)
 - 4-1 MSE
 - 4-2 MAE
 - 4-3 경사하강법
5. 딥러닝
 - 5-1 심층 신경망
 - 5-2 순전파 알고리즘
 - 5-3 역전파 알고리즘
6. 헛갈리는 부분 정리

1. 머신 러닝 (Machine Learning)

머신러닝은 인공지능의 한 분야로 컴퓨터가 명시적으로 프로그래밍 되지 않아도 데이터로부터 패턴을 스스로 학습하여 특정 작업을 수행하는 방법 혹은 기술이다. 전통적인 프로그래밍은 사람이 직접 규칙과 알고리즘을 명시적으로 작성해주었다면 머신러닝은 데이터로부터 모델이 스스로 규칙과 알고리즘 학습한다. 예를 들어 고양이 이미지와 강아지 이미지를 분류하는 문제가 있다고 해보자. 이 문제를 해결하기 위해서 전통적인 프로그래밍은 고양이와 강아지를 구별할 수 있도록 고양이와 강아지의 차이를 코드로 하나하나 다 작성하여 문제를 해결한다. 하지만 머신러닝은 고양이와 강아지의 차이를 학습하고 학습한 것을 토대로 고양이와 강아지를 분류하여 문제를 해결한다. 또한 머신러닝을 사용한다면 고양이와 강아지가 아니라 사자와 호랑이를 분류하는 것도 가능하다. 왜냐하면 두 이미지의 차이를 스스로 학습하기 때문에 사자와 호랑이를 분류하는 문제에서도 두 동물의 차이를 학습하고 분류하기 때문이다. 이렇게 머신러닝은 전통적인 프로그래밍 방식보다 해결할 수 있는 문제가 많고 데이터로부터 패턴을 학습하여 분류, 인식 등 자동화에 유용하기 때문에 우리가 머신러닝을 사용하는 것이다. 이 머신러닝은 지도 학습, 비지도 학습, 강화 학습으로 나눌 수 있다.

1-1 지도 학습 (Supervised Learning)

지도 학습이란 이란 정답이 알고있는 데이터를 통해 컴퓨터가 스스로 규칙을 학습하도록 만드는 방법 혹은 기술이다. 1) 어떠한 문제가 있을 때 우리가 그 문제에 대한 답을 알고 있고 인공지능이 그것을 알아낼 수 있도록 한다. 위의 고양이, 강아지 이미지를 예시로 들면 우리가 이미지와 이미지가 고양이, 강아지 중 어떤 것에 해당하는지에 대한 정답을 같이 입력한다. 인공지능이 이미지와 정답의 상관 관계를 찾고 학습한 다음 고양이와 강아지 이미지를 분류할 있게 되는 것이다.

1-2 비지도 학습 (Unsupervised Learning)

비지도 학습은 지도 학습의 반대 개념이다. 지도 학습에서는 정답이 있는 데이터를 입력해주었다면 비지도 학습에서는 정답을 모르는 데이터를 입력하여 데이터의 숨겨진 구조나 패턴을 찾아는 방법 기술이다. 2) 지도 학습은 문제를 해결하는 것이 목적이었다면 비지도 학습의 목적은 데이터 자체에서 정보를 추출하는 것이 목적이다. 고양이, 강아지 이미지를 입력하면 비지도 고양이, 강아지 이미지의 특징을 추출하여 학습한다.

1-3 강화 학습(Reinforcement Learning)

강화학습이란 에이전트가 계속해서 새로운 행동을 시도하고 행동을 통해 얻은 결과로부터 보상을 극대화할 수 있는 최적의 행동을 취하도록 학습하는 방법 혹은 기술이다.3) 쉽게 말하면 어떠한 문제가 있을 때 문제를 해결하기 위한 여러 가지 해결법을 계속해서 시도해보고 그 문제를 해결할 수 있는 확률이 가장 높은 방법을 학습하는 것이다. 휴머노이드 보행 기술을 예로 들면 휴머노이드 로봇이 시뮬레이션 환경에서 계속해서 보행을 하면서 보상 함수를 통해 결과를 얻고 이 결과가 최적이 되는 행동을 학습하여 스스로 균형을 잡고 걷는 법을 익히는 것이다.

2. 퍼셉트론

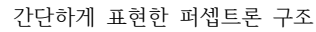
퍼셉트론은 다수의 입력으로부터 하나의 결과를 내보내는 알고리즘이다.4) 인간의 신경 세포인 뉴런의 가지돌기에서 여러 신호를 받아들이고 자신의 용량을 넘어서면 축삭 돌기를 통해 신호를 전달하는 뉴런의 작동 방식을 모사하여 만들어졌다.

1) 지도 학습

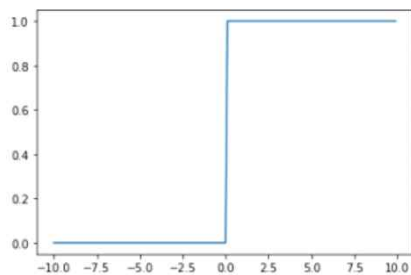
2) 비지도 학습 <https://wikidocs.net/238860>

3) 강화학습 <https://wikidocs.net/236199>

4)[신경망 이론] 퍼셉트론 <https://roytravel.tistory.com/195>



3-1 step 함수(계단 함수)



$$f(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases}$$

step 함수가 퍼셉트론에 사용되는 활성화 함수이다. 입력값 또는 가중합이 임계값을 넘기면 1, 그렇지 않으면 0을 출력하는 함수이다.

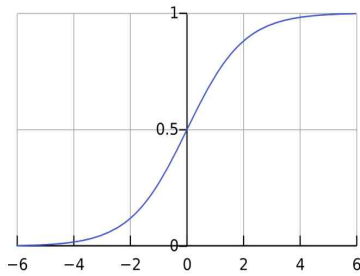
PyTorch에서는 Step 함수를 위한 nn.Step() 클래스가 따로 제공되지 않는다. 따라서 조건식을 이용하여 직접 구현할 수 있다.

```
1 import torch
2
3 x = torch.tensor([-1.0, 0.0, 1.0])
4
5 y = (x >= 0).float()
6
```

`x >= 0`을 이용하여 입력값이 0 이상이면 True, 0보다 작으면 False가 되도록 한다. 이후 `.float()`를 사용하여 True는 1, False는 0으로 변환한다.

3-2 sigmoid 함수

시그모이드(sigmoid) 함수는 1과 0 두 가지 분류인 2진 분류 문제에 적합한 함수이다. 시그모이드는 출력값이 최대가 1 최소가 0이고 중앙값은 0.5인 활성화 함수이다.



$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

PyTorch에서는 Sigmoid 함수를 `torch.sigmoid()` 함수 또는 `nn.Sigmoid()` 클래스로 사용할 수 있다.

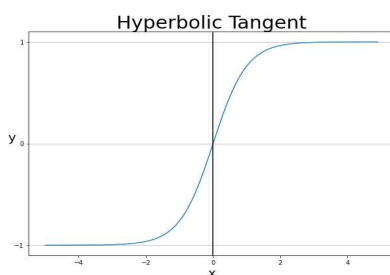
```

1 import torch
2 import torch.nn as nn
3
4 x = torch.tensor([-1.0, 0.0, 1.0])
5
6 sigmoid = nn.Sigmoid()
7 y = sigmoid(x)

```

`nn.Sigmoid()`를 이용하여 Sigmoid 활성화 함수 객체를 생성하고, 입력값 `x`를 넣어 각 값에 Sigmoid 함수를 적용한다. 출력값은 0과 1 사이의 값으로 변환된다.

3-3 Tanh 함수⁶⁾



$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

하이퍼볼릭 탄젠트(Hyperbolic) 함수는 대학수학및연습1에서 배웠던 쌍곡선 탄젠트 함수이다. 위의 시그모이드 함수와 형태가 매우 비슷하다. 하지만 시그모이드 함수와 몇 가지 차이점이 있다. 먼저 출력값의 범위가 다르다. 시그모이드 함수는 출력값이 최대 1, 최소 0이었다. 하이퍼볼릭 탄젠트 함수의 출력값은 최대 1, 최소 -1이다. 또한 중앙값도 0이다.

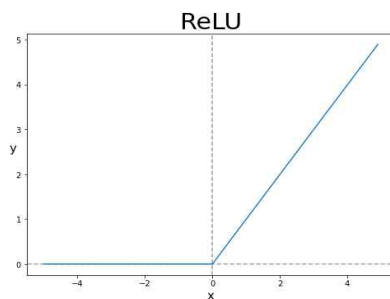
6) 활성화 함수 - 하이퍼볼릭 탄젠트 함수 (<https://gooopy.tistory.com/54>)

PyTorch에서는 Tanh 함수를 `torch.tanh()` 함수 또는 `nn.Tanh()` 클래스로 사용할 수 있다.

```
1 import torch
2 import torch.nn as nn
3
4 x = torch.tensor([-1.0, 0.0, 1.0])
5
6 tanh = nn.Tanh()
7 y = tanh(x)
```

`nn.Tanh()`를 이용하여 Tanh 활성화 함수 객체를 생성하고 입력값 x 에 적용한다. 출력값은 -1과 1 사이의 값으로 변환된다.

3-4 ReLU 함수⁷⁾



$$ReLU(x) = \begin{cases} x & (x > 0) \\ 0 & (x \leq 0) \end{cases}$$

렐루(ReLU) 함수가 등장하기 전 심층 신경망(DNN)에는 시그모이드 함수가 주로 사용되었지만 렐루 함수가 등장한 후에는 렐루 함수가 주로 사용되고 있다. 렐루 함수는 입력값이 0보다 크면 그대로 자기 자신을 출력하고 입력값이 0보다 작다면 0을 출력하는 활성화 함수이다. 그래서 $f(x) = \max(0, x)$ 라고 표현하기도 한다.

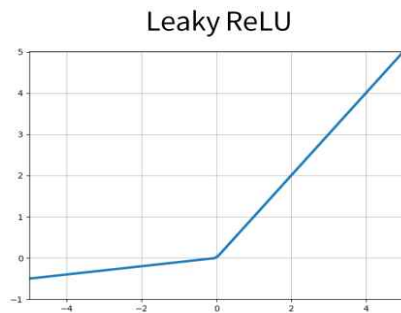
PyTorch에서는 ReLU 함수를 `torch.relu()` 함수 또는 `nn.ReLU()` 클래스로 사용할 수 있다.

```
1 import torch
2 import torch.nn as nn
3
4 x = torch.tensor([-1.0, 0.0, 1.0])
5
6 relu = nn.ReLU()
7 y = relu(x)
```

7) 활성화 함수 - 렐루 함수 (<https://gooopy.tistory.com/55>)

nn.ReLU()를 이용하여 ReLU 활성화 함수를 생성한다. 입력값이 0보다 작으면 0으로 출력하고 0보다 크면 입력값을 그대로 출력한다.

3-5 Leaky ReLU 함수⁸⁾



$$\text{Leaky ReLU}(x) = \max(0.1x, x)$$

리키 렐루 함수(Leaky ReLU) 함수는 입력값이 0보다 크면 입력값을 그대로 출력하고 0보다 같거나 작으면 0.1 * 입력값을 출력한다.

PyTorch에서는 Leaky ReLU 함수를 nn.LeakyReLU() 클래스로 사용할 수 있다.

```
1 import torch
2 import torch.nn as nn
3
4 x = torch.tensor([-1.0, 0.0, 1.0])
5
6 leaky_relu = nn.LeakyReLU(negative_slope=0.01)
7 y = leaky_relu(x)
```

nn.LeakyReLU()를 이용하여 Leaky ReLU 활성화 함수를 생성한다. negative_slope=0.01은 입력값이 음수일 때 완전히 0으로 만들지 않고 입력값에 0.01을 곱하여 작은 값으로 출력하도록 설정한다.

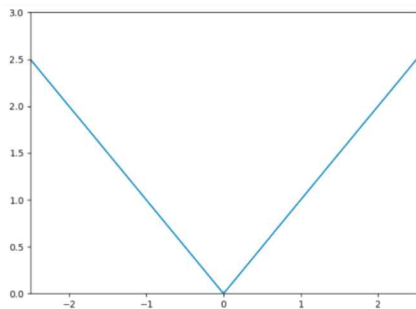
4. 비용함수(Loss function, cost function)⁹⁾

비용 함수는 머신 러닝과 딥러닝에서 모델의 성능을 측정하는데 사용되는 함수이다. 모델의 출력값(예측값)과 실제값 간의 차이를 수치화하여 모델의 오차율을 구한다. 이 오차를 줄이

8) <https://velog.io/@lighthouse97/%EA%B8%B0%EC%9A%B8%EA%B8%B0-%EC%86%8C%EC%8B%A4-%EB%AC%B8%EC%A0%9C%EC%99%80-ReLU-%ED%95%A8%EC%88%98>

기 위해 예측 값이 실제 값과 얼마나 유사한지 판단하는 기준이 필요한데 그것이 손실 함수이다. 손실 함수의 값이 작을수록 오차가 작은 것이고 클수록 오차가 큰 것이다. 머신러닝과 딥러닝은 손실 함수의 값을 줄이는 방향으로 학습을 진행한다. 이 과정을 우리는 최적화 (Optimization) 한다. 라고 말한다. 대표적인 손실 함수로는 MAE 손실 함수와 MSE 손실 함수가 있다.

4-1 MAE (Mean Absolute Error, 평균 절대 오차)



$$MAE = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$$

평균 절대 오차는 위의 식과 같이 오차 합 of 절대값을 취한 형태이다. $\hat{y}$ 은 인공지능 모델이 예측한 값이며 y 는 실제 값이다. 오차에 절대값을 취하는 이유는 오차 값이 음수이면 오차들을 더했을 때 오차의 절대적인 값이 적어져 정확한 오차의 총합을 알 수 없기 때문이다.

PyTorch에서는 MAE 손실 함수를 `nn.L1Loss()` 클래스로 사용할 수 있다.

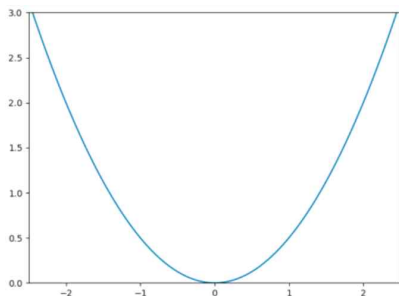
```

1 import torch
2 import torch.nn as nn
3 prediction = torch.tensor([2.0, 3.0, 4.0])
4 target = torch.tensor([1.0, 3.0, 5.0])
5 mae = nn.L1Loss()
6 loss = mae(prediction, target)
7

```

`nn.L1Loss()`는 예측값과 실제값의 차이에 절대값을 적용하고, 그 값들의 평균을 계산하여 손실값을 구한다. PyTorch에서는 MAE를 `L1Loss`라는 이름으로 제공한다.

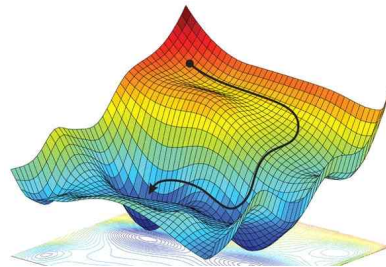
4-2 MSE (Mean Squared Error, 평균 제곱 오차)



$$MSE = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

평균 제곱 오차는 위의 식과 같이 오차 제곱의 합의 평균 형태이다. $\hat{y}$ 은 인공지능 모델이 예측한 값이며 y 는 실제 값이다. 이 오차를 제공하는 이유는 MAE, 평균 절대 오차에서 절대값을 취한 이유와 같다. 평균 제곱 오차에서는 오차를 제곱을 하게 됨으로 큰 유라가 가장되는 장점이 있다.

최적화 과정에서는 손실 함수의 값이 가장 작은 값이 되는 해를 찾아야 하는데 위의 MAE, MSE 모두 2차 함수 형태이다. 그러므로 미분했을 때 0이 되는 지점 즉 기울기가 0이 지점이 가장 작은 손실 함수의 값을 가지는 최적의 해이다. 하지만 실제 손실 함수의 그래프는 2차 함수 형태가 아니라 고차원 함수이다.



실제 고차원 데이터를 다룰 때 손실 함수의 그래프 모양이다. 눈으로 봐서는 절대로 몇 차원인지 알 수 없고 위의 2차 함수와 다르게 미분했을 때 0이 되는 지점을 찾는 것 또한 매우 어려울 것이다. 그래서 고차원 손실 함수의 최소값을 찾기 위해 고안된 방법이 어떠한 지점에서 조금씩 움직이며 최소값을 찾아 나가는 경사하강법이다.

PyTorch에서는 MSE 손실 함수를 `nn.MSELoss()` 클래스로 사용할 수 있다.

```

1 import torch
2 import torch.nn as nn
3 prediction = torch.tensor([2.0, 3.0, 4.0])
4 target = torch.tensor([1.0, 3.0, 5.0])
5 mse = nn.MSELoss()
6 loss = mse(prediction, target)
7

```

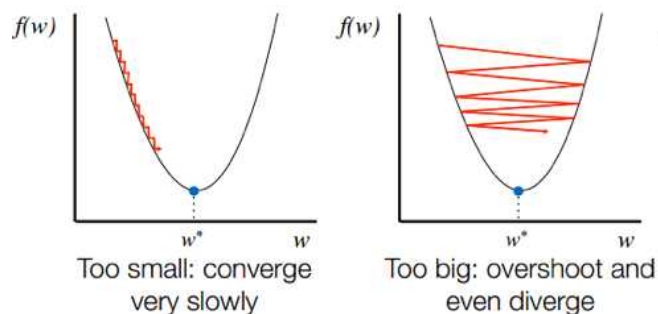
prediction은 모델이 예측한 값이고, target은 실제 정답값이다. nn.MSELoss()는 예측값과 실제값의 차이를 제공한 뒤 평균을 계산하여 손실값을 구한다.

4-3 경사하강법(gradient descent)

경사하강법은 주어진 함수에서 극소점(손실 함수의 값이 최소값이 되는 해)을 찾기 위해 기울기가 최소가 되는 지점을 찾아가는 방법이다. 10) 경사하강법은 안개가 낀 산에서 하산할 때의 상황으로 많이 비유하여 설명한다. 안개가 낀 산에서는 아무것도 보이지 않기 때문에 하산할 때 산 비탈길의 기울기가 가파른 방향을 찾고 그 방향을 내려가 산을 하산한다. 경사하강법도 똑같다. 어느 한 지점에서 함수의 기울기가 감소하는 방향을 찾고 그 방향을 이동해 함수의 극소점을 찾는다. 좀 저세히 설명하면 현재 지점의 기울기를 구하고 기울기가 가파른 방향으로 이동할 방향을 정한다. 그리고 한 번 이동할 때 마다 얼마나 이동할 것인지 step size를 정하여 곱해서 그만큼 이동한다. 위의 과정을 함수의 극소점을 찾을 때 가지 반복한다.

$$x_{\text{새로운 위치}} = x_{\text{현재 위치}} - \eta \frac{df}{dx}(x_{\text{현재 위치}})$$

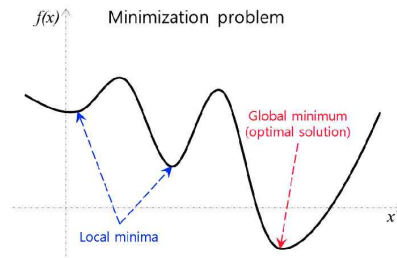
식을 나타내면 위처럼 표현한다. 위 식은 현재 위치에서 현재 위치에서 현재 위치의 기울기를 빼주어 새로운 위치를 정한다. 여기서 step size 즉 학습률(Learning rate)은 η 이다. 이 학습률은 위에서 설명했듯이 기울기를 따라 얼마나 이동할지 결정한다. 이 학습률이 너무 작으면 점을 찾는데 오랜 시간일 필요하고 반대로 학습률이 너무 크면 최저점을 지나쳐버리는 현상이 발생하여 제대로 학습을 하지 못하는 문제가 발생한다. 밑의 사진은 학습률이 매우 작을 때와 매우 클 때 2차원 손실 함수에서 발생하는 현상을 보여주는 이미지이다.



제대로 된 학습을 위해서는 적절한 학습률을 찾아야 한다. 11) 경사하강법의 한계점 또한 존재한다. 한계점은 아래와 같다.

10) 경사하강법 - <https://dotiromooook.tistory.com/25>

11) Date science-https://yngie-c.github.io/deep%20learning/2020/03/10/neural_net_training/



이 한계점은 학습률을 매우 작게했을 때 많이 발생하는 한계이다. 위의 이미지처럼 경사하강법을 이용해 최적화 시 Global minimum(전역 극소점)을 찾아야 하는데 최적화 과정에서 Local minima(지역 극소점)에 빠지는 경우이다. 이렇게 되면 최적화 과정에서 제대로 된 해를 찾지 못하고 잘못된 해를 찾게 된다.

PyTorch에서는 경사하강법을 사용할 때 `torch.optim.SGD()`를 사용할 수 있다.

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 model = nn.Linear(1, 1)
5 optimizer = optim.SGD(model.parameters(), lr=0.01)
```

`optim.SGD()`는 경사하강법을 이용하여 모델의 가중치와 편향을 수정하는 Optimizer이다. `model.parameters()`에는 모델이 학습해야 할 가중치와 편향이 들어가고 `lr=0.01`은 위에서 설명한 학습률이다. 가중치를 얼마나 크게 수정할 것인지를 결정한다.

5. 딥러닝

5-1 심층 신경망(Deep Neural Network, DNN)

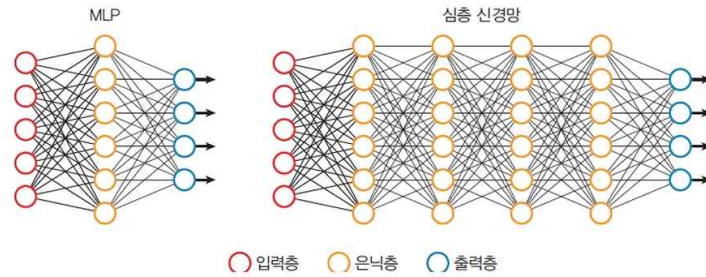
심층 신경망은 인공신경망에 은닉층의 개수를 증가시킨 것이다. 흔히 우리가 딥러닝이라고 부른다. ¹²⁾

12)

심층

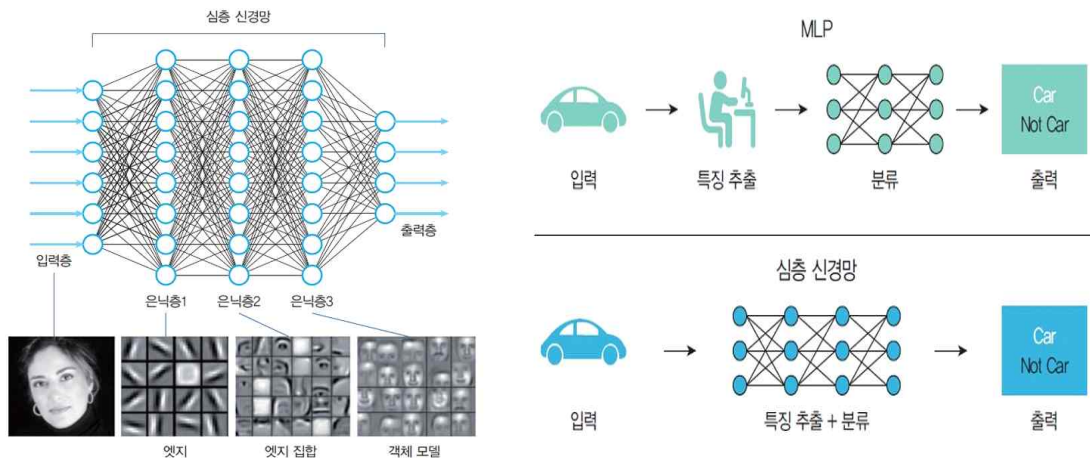
신경망-

<https://velog.io/@tjddnr209/%EC%8B%AC%EC%B8%B5%EC%8B%A0%EA%B2%BD%EB%A7%9D>



인공신경망과 심층 신경망의 차이

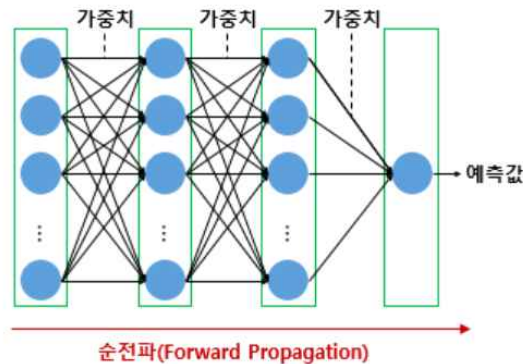
심층 신경망은 기본적인 인공 신경망에 비해 은닉층의 개수가 많기 때문에 더욱 복잡한 데이터를 학습할 수 있다.



위의 그림과 같이 심층 신경망은 여러 개의 은닉층으로 구성되어 있어 입력된 데이터의 특징을 단계적으로 학습할 수 있다. 예를 들어 사람의 얼굴 이미지가 입력되면 앞쪽 은닉층에서는 선이나 모양과 같은 비교적 단순한 특징을 학습하고, 그다음 은닉층에서는 눈, 코, 입과 같은 더 복잡한 특징을 학습할 수 있다. 그리고 더 깊은 은닉층에서는 앞에서 학습한 여러 특징들을 종합하여 사람의 얼굴을 구분하거나 분류하는 데 사용한다.

심층 신경망의 중요한 특징 중 하나는 사람이 직접 데이터의 특징을 모두 만들어서 입력하지 않아도 된다는 점이다. 기존의 머신 러닝 방법에서는 사람이 데이터에서 필요한 특징을 직접 선택하거나 추출하여 모델에 입력하는 경우가 많았다. 하지만 심층 신경망에서는 원본 데이터를 입력하면 여러 은닉층을 거치면서 필요한 특징을 스스로 학습하고 학습한 특징을 이용하여 최종적으로 데이터를 분류하거나 예측할 수 있다. 따라서 은닉층이 깊어질수록 단순한 특징부터 복잡한 특징까지 단계적으로 학습할 수 있다는 장점이 있다.

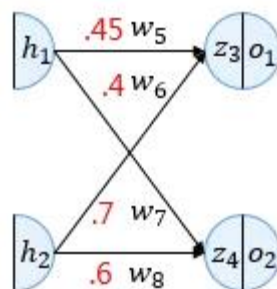
5-2 순전파 알고리즘(forward propagation) 13)



순전파 알고리즘은 입력 데이터가 모델의 입력층에서 출력층 방향으로 순서대로 전달되는 과정이다. 각 층에서는 입력값에 가중치를 곱하고 편향을 더해 가중합을 계산한 뒤 이 값을 활성화 함수에 입력하여 출력값을 만든다. 이렇게 계산된 출력값은 다음 은닉층의 입력값으로 사용되며 같은 과정이 반복되어 최종적으로 출력층에서 예측 결과가 만들어진다. 즉 순전파 알고리즘은 앞에서 설명한 퍼셉트론의 계산 과정이 여러 층에서 반복되는 것이라고 볼 수 있다. 하나의 퍼셉트론에서는 활성화 함수를 거쳐 나온 값이 최종 출력이 될 수 있지만 다층 신경망에서는 이 출력값이 다음 층의 입력값으로 전달되어 다시 가중합과 활성화 함수 계산을 거치게 된다.

5-3 역전파 알고리즘(back propagation) 14)

역전파 알고리즘은 이름에서도 알 수 있듯이 순전파 알고리즘의 과정을 역으로 수행하는 것이다. 이 역전파 알고리즘을 사용하는 이유는 순전파 알고리즘에서 구한 예측 값과 실제값으로부터 오차를 계산하고 오차로부터 가중치와 편향을 업데이트하는 과정이다. 즉 모델의 성능을 올려주는 과정이라고 생각하면 된다.



w_5 를 예시로 역전파 알고리즘의 과정을 설명해보겠다. 일단 예측값과 실제값의 차이 오차를 계산해야 한다. 오차는 임의로 $E(\text{Error})$ 라고 가정하겠다. 이제 역으로 가중치가 이 오차에 얼

13) 순전파 알고리즘 - <https://yc7764.tistory.com/27>

14) 역전파 알고리즘 - <https://wikidocs.net/37406>

마나 영향을 미쳤는지 계산해야한다. 이 과정이 역전파 알고리즘의 핵심이다.

$$\frac{\partial E}{\partial w_5} = \frac{\partial E}{\partial \sigma_1} \times \frac{\partial \sigma_1}{\partial z_3} \times \frac{\partial z_3}{\partial w_5}$$

E에 가중치가 얼마나 영향을 미쳤는지를 수식을 나타내면 좌변과 같고 좌변을 풀어쓴 식이 우변과 같다. 우변의 각 항에 대해 설명하면 활성화 함수를 거쳐 출력된 σ_1 이 E에 얼마나 영향을 미치는 가를 뜻하는 항이다. 두 번째 항은 z_3 가 활성화 함수의 출력값 σ_1 에 얼마나 영향을 미치는 가를 뜻하는 항이다. 세 번째 항은 가중치 w_5 가 활성화 함수 입력값에 얼마나 영향을 미치는 가를 뜻하는 항이다.

이 세 항을 각각 미분한 후 모두 곱하면 가중치가 최종 오차에 미치는 전체적인 영향을 구할 수 있다. 이렇게 구한 값이 가중치에 대한 기울기이다.

$$x_{\text{새로운 위치}} = x_{\text{현재 위치}} - \eta \frac{dE}{dw_5}(x_{\text{현재 위치}})$$

기울기를 구한 후에 앞에서 설명한 경사하강법을 이용하여 이 기울기를 바탕으로 가중치를 수정한다, 같은 방법으로 출력층에서 시작하여 은닉층 방향으로 각 가중치의 기울기를 차례대로 계산하고 가중치를 수정한다. 이처럼 최종 오차를 기준으로 출력층에서 입력층 방향으로 거슬러 올라가며 각 가중치의 기울기를 계산하는 과정을 역전파 알고리즘이라고 한다.

6. 헛갈리는 부분 정리

1. 퍼셉트론과 인공 신경망의 관계

처음에는 퍼셉트론과 인공 신경망이 같은 개념인지 헛갈렸다. 퍼셉트론은 여러 입력값을 받아 각각의 가중치를 곱하고 편향을 더한 뒤 활성화 함수를 거쳐 하나의 출력값을 만드는 기본적인 인공 뉴런이다. 이러한 퍼셉트론을 여러 개 연결하여 층을 구성하면 인공 신경망을 만들 수 있다. 따라서 퍼셉트론은 인공 신경망 자체라기보다 인공 신경망을 구성하는 기본 단위라고 이해할 수 있다. 퍼셉트론을 여러 층으로 연결한 구조를 다층 퍼셉트론이라고 하며 은닉층을 여러 개 쌓아 깊게 만든 인공 신경망을 심층 신경망(DNN)이라고 한다. 즉 퍼셉트론, 다층 퍼셉트론, 인공 신경망, 심층 신경망의 관계를 다음과 같이 정리할 수 있다. 퍼셉트론은 신경망을 구성하는 기본 뉴런이고 여러 퍼셉트론을 연결하면 다층 퍼셉트론을 만들 수 있다. 다층 퍼셉트론은 인공 신경망의 한 종류이며 이러한 신경망의 은닉층을 여러 개 쌓으면 심층 신경망이 된다.

2. 활성화 함수의 역할

처음에는 활성화 함수가 단순히 퍼셉트론의 출력을 결정하는 함수라고만 생각했다. 퍼셉트론에서는 Step 함수를 사용하여 임계치를 기준으로 0 또는 1을 출력하기 때문에 이러한 이해도 맞지만 심층 신경망에서는 활성화 함수가 더 중요한 역할을 한다. 신경망에서 계산은 선형적인 계산이다. 이 값에 ReLU, Sigmoid, Tanh와 같은 비선형 활성화 함수를 적용하면 신경망에 비선형성을 추가할 수 있다. 활성화 함수가 없다면 여러 개의 선형층을 쌓더라도 결국 하나의 선형식으로 표현할 수 있다. 따라서 복잡한 데이터의 패턴을 학습하기 어렵다. 즉 활성화 함수는 가중합으로 계산된 값을 비선형적으로 변환하여 신경망이 더 복잡한 데이터를 학습할 수 있도록 해주는 역할을 한다.

3. 손실 함수와 역전파의 관계

처음에는 손실 함수의 크기를 줄이는 과정 자체가 역전파라고 생각했다. 하지만 손실 함수는 모델의 예측값과 실제 정답값이 얼마나 차이가 나는지를 계산하는 역할을 한다. 그리고 역전파는 이 손실값을 기준으로 각 가중치가 손실에 얼마나 영향을 미쳤는지를 계산하는 과정이다. 즉 역전파는 손실을 직접 줄이는 것이 아니라 기울기(Gradient)를 계산한다. 이 Gradient를 이용하여 실제로 가중치를 수정하는 과정은 경사하강법과 같은 Optimizer가 담당한다. 따라서 각 역할을 다음과 같이 구분할 수 있다. 손실 함수는 모델의 오차를 계산하고 역전파는 각 가중치에 대한 Gradient를 계산하며 경사하강법은 계산된 Gradient를 이용하여 실제 가중치를 수정한다.